

Polling Server (incl. Introduction to Fixed Priority Servers)

Md Azadul Islam

Student ID: 2220070

Real-Time Systems

Hamm-Lippstadt University of Applied Sciences

Seminar Paper

Abstract—Real-time systems usually combine predictable periodic activities with irregular aperiodic requests. If aperiodic work is executed without control, it can disturb hard real-time deadlines; if it is executed only in background time, response time may become unacceptable. The Polling Server is the simplest fixed-priority server mechanism for this problem: it behaves like an ordinary periodic task, checks at fixed instants whether an aperiodic request is waiting, serves it if so, and otherwise discards its budget for that cycle. This paper explains the capacity and polling idea behind the server, presents a compact UPPAAL model of the main capacity-loss mechanism, compares the Polling Server with related fixed-priority servers, and evaluates its assumptions, overheads and application limits. The conclusion is that the Polling Server is useful but limited: it is the easiest of the fixed-priority servers to implement and analyse, but it is also the only one that never preserves unused capacity, which gives it the worst guaranteed and average aperiodic response time of the group.

Index Terms—Real-time systems, fixed-priority scheduling, aperiodic tasks, Polling Server, scheduling servers, UPPAAL.

I. INTRODUCTION

In a real-time system, a result that arrives too late can be as harmful as a result that is simply wrong [2, Sec. 1.2]. Many embedded systems run periodic tasks, such as control loops or sensor readings, that repeat at a fixed rate and are usually scheduled with algorithms such as Rate Monotonic [1] [2, Sec. 2.2.1]. At the same time, the same system often has to react to aperiodic events, such as alarms, operator commands or communication messages, that can arrive at any time [2, Sec. 2.2.1].

This mix is the problem looked at in this paper. Aperiodic requests should be answered quickly, but they must not take processor time away from the periodic tasks [2, Ch. 5]. One simple option is to run aperiodic work only when the processor is otherwise idle; this is safe, but the response time can become very poor [2, Sec. 5.2]. Another option is to give aperiodic work a high priority directly, but this can put the periodic deadlines at risk [2, Sec. 5.1]. The Polling Server sits between these two extremes: it reserves a small, fixed slice of processor time for aperiodic work and checks, at regular instants, whether that slice is actually needed [2, Sec. 5.3] [4].

This paper explains how the Polling Server works, compares it with the related fixed-priority servers, shows its main behaviour with a small UPPAAL model, and discusses where it is actually useful in an embedded system.

II. BACKGROUND AND PROBLEM SETTING

A periodic task releases jobs at regular intervals and is described by a period, a relative deadline and a worst-case execution time [2, Sec. 2.2.1]; this model underlies most control and monitoring functions and is the basis of fixed-priority scheduling under Rate Monotonic (RM) [1], [2, Sec. 4.3]. Aperiodic tasks, in contrast, consist of an infinite sequence of jobs whose activations are not regularly interleaved – their exact arrival times are not known in advance, and an aperiodic task with a guaranteed minimum inter-arrival time is called a sporadic task [2, Sec. 2.2.1, Fig. 2.5]. Either way, such requests cannot be folded into the offline analysis of the periodic task set.

Background scheduling, where aperiodic work runs only when no periodic task is ready, is the simplest way to combine the two [2, Sec. 5.2]. It cannot hurt the periodic tasks, but the resulting aperiodic response time can become very large, especially when the periodic load is high. A server improves on this by reserving a controlled amount of processor capacity for aperiodic work; the challenge is to manage that capacity so that aperiodic response improves without breaking the schedulability of the periodic task set [2, Ch. 5].

The Polling Server is normally discussed in exactly this fixed-priority preemptive setting: the server itself is given a priority and is analysed as part of the workload, like any other periodic task [2, Ch. 5]. This matters because the server must not behave like an uncontrolled high-priority task – its budget and polling rule must bound the processor demand it can create.

III. CORE IDEA OF THE POLLING SERVER

The Polling Server is characterised by a period T_{ps} and a capacity C_{ps} [2, Sec. 5.3]. It is inserted into the periodic task set and given a priority by the same rule as the periodic tasks, as if it were itself a periodic task with parameters (C_{ps}, T_{ps}) [2, Sec. 5.3]. Every T_{ps} time units it wakes up and checks the aperiodic queue:

- if at least one request is waiting, it serves pending work for up to C_{ps} time units;
- if nothing is waiting, it suspends immediately, and whatever was left of C_{ps} for that cycle is lost – the unused capacity is discharged to the periodic tasks instead of being carried over to the next activation [2, Sec. 5.3].

The share of the CPU reserved this way is the server bandwidth $U_{ps} = C_{ps}/T_{ps}$ [2, Sec. 5.3.1]. The worked example used throughout Buttazzo’s presentation of the Polling Server uses $C_{ps} = 2$ [2, Fig. 5.3], the same capacity reused for the UPPAAL model in Section IV.

Because the server occupies the CPU exactly like a periodic task with (C_{ps}, T_{ps}) , the same kind of schedulability tests used for periodic tasks under Rate Monotonic can be reused for the server as well [1] [2, Sec. 5.3.1]. Buttazzo gives both a simple utilisation-based test and a tighter, hyperbolic test for this case, and also derives the worst-case response time of a periodic task when a Polling Server is present [2, Sec. 5.3.1, Eq. (5.4)–(5.6)]. Any task with a lower priority than the server simply sees it as one more interfering task of this kind.

Given a periodic task set, Buttazzo also explains how to choose C_{ps} and T_{ps} : the largest server utilisation that keeps the periodic task set schedulable is computed first, and the server period is then normally set equal to the period of the fastest periodic task, so that the server gets the highest priority without unnecessarily fragmenting its capacity [2, Sec. 5.3.2, Eq. (5.7)–(5.8)].

The distinctive feature of the Polling Server – and also its main weakness – is that capacity is never preserved [2, Sec. 5.3]. If a request arrives an instant after a poll has found the queue empty, it has to wait for the next poll, up to T_{ps} time units, before the server even looks at it. Buttazzo also gives a guarantee test and a response-time formula for a single aperiodic job served this way when the server has the highest priority [2, Sec. 5.3.3, Eq. (5.9), Fig. 5.6]. In all cases, the worst-case response time is on the order of $T_{ps} + C_{ps}$, independent of how idle the processor actually was when the request arrived. The Deferrable Server, Priority Exchange and the Sporadic Server change exactly this replenishment rule, as discussed in Section V.

IV. UPPAAL MODEL OF THE MAIN MECHANISM

To demonstrate the main mechanism of the Polling Server, a small UPPAAL model was built using timed automata [8]; UPPAAL’s model checker can decide whether a given property holds in every reachable state, or in at least one [9], [10]. The model is not a generic, parametric server – it is a formal demonstration of the capacity-loss rule and of the resulting worst-case waiting time discussed in Section III.

The model contains three templates, Job1, Job2 and PollingServer, sharing a constant $BUDGET = 2$ (i.e. $C_{ps} = 2$ ms), clocks g , s and $idle$, an integer queue, and the booleans $job1_wait$, $job2_wait$, $served1$, $served2$ and $budget_destroyed$.

The PollingServer template (Fig. 1) is a chain of wait locations and committed decision locations (the “C” locations, left in zero time) whose guards and updates implement exactly the polling, serving and budget-loss logic described in Section III. Polls happen at $g=2, 5, 10$; ServeJob1 and ServeJob2 carry the invariant $s \leq BUDGET$, and the $Idle \rightarrow Idle$ self-loop only exists so that UPPAAL does not report a deadlock at the end of this single-shot scenario.

TABLE I
DETERMINISTIC TIMELINE PRODUCED BY THE MODEL

Time	Event
0 ms	Model starts, queue empty.
1 ms	Job1 arrives ($queue=1$, $job1_wait=true$).
2 ms	First poll: queue non-empty $\rightarrow$ ServeJob1 starts.
2–4 ms	Job1 is served ($budget = 2$ ms).
4 ms	Job1 finished ($served1=true$).
5 ms	Second poll: queue empty $\rightarrow$ $budget_destroyed=true$.
6 ms	Job2 arrives, must wait – budget already lost.
10 ms	Third poll: queue non-empty $\rightarrow$ ServeJob2 starts.
10–12 ms	Job2 is served ($budget = 2$ ms).
12 ms	Job2 finished ($served2=true$), model enters Idle.

Job1 (Fig. 2) and Job2 (Fig. 3) each generate one aperiodic request at a fixed time, $g=1$ and $g=6$ respectively (the location BeforeA1 in Fig. 2 is a spelling typo for BeforeA1 in the model file). These two times were chosen on purpose: Job1 arrives before the first poll and is served immediately, while Job2 arrives just after the poll that empties the queue, so the worst-case pattern from Section III actually occurs in the model.

Running the model produces the timeline in Table I: Job1 arrives at $t=1$ and is served from $t=2$ to $t=4$; at $t=5$ the second poll finds the queue empty and $budget_destroyed$ becomes true; Job2 arrives at $t=6$ but has to wait until the third poll at $t=10$, and is served from $t=10$ to $t=12$.

Four reachability ($E\Diamond$) properties were checked, all satisfied (Fig. 4): $served1$ (Job1 can be fully served), $budget_destroyed$ (the capacity-loss state is actually reached), $PollingServer.WaitThirdPoll \ \&\& \ job2_wait$ (Job2 is stuck waiting for the third poll while it has already arrived – the model-checked version of the worst case), and $served2$ (the delayed request is eventually served). Together these four properties show that the worst-case waiting pattern from Section III is an execution the model actually reaches, not only an upper bound on paper.

V. RELATED APPROACHES AND TRADEOFFS

Buttazzo treats background scheduling, the Polling Server, the Deferrable Server, Priority Exchange, the Sporadic Server and Slack Stealing as the fixed-priority server approaches for hybrid workloads consisting of hard periodic and soft or firm aperiodic activity [2, Ch. 5]. They all address the same basic problem as the Polling Server but differ in how reserved capacity is preserved, consumed and replenished.

Background scheduling is the simplest and protects periodic jobs perfectly, since it only uses idle time, but its aperiodic response time is poor, especially under high periodic load [2, Sec. 5.2]. The Deferrable Server keeps the same period and budget as the Polling Server but preserves unused capacity until the end of the period, replenishing it to full value at every period boundary [2, Sec. 5.4] [3]; a late request can then be served immediately, at the cost of possible back-to-back execution of two full budgets and a more pessimistic interference analysis. Priority Exchange takes a different approach again: instead of discarding unused capacity, it trades it

TABLE II
QUALITATIVE COMPARISON OF FIXED-PRIORITY APERIODIC SERVICE
APPROACHES

Approach	Response	Main limitation
Background	Poor	Only uses idle time
Polling Server	Poor	Capacity lost at empty polls
Deferrable Server	Good	More complex interference
Priority Exchange	Good	Bookkeeping across priorities
Sporadic Server	Good	Replenishment accounting
Slack Stealing	Very good	High computation cost

Based on [2, Sec. 5.2–5.9]; qualitative, not a universal ranking.

uses that slack immediately [2, Sec. 5.7] [5], giving the best average response time of the group at the cost of continuously tracking the remaining execution of every periodic task.

Buttazzo’s simulation results show that the Deferrable Server, Priority Exchange and Sporadic Server all substantially reduce the average aperiodic response time compared with background and polling service [2, Sec. 5.9, Fig. 5.24], with the Sporadic Server falling slightly behind the other two precisely because its capacity is restored only T_{ps} time units after consumption, rather than at the next period boundary [2, Sec. 5.9]. Slack Stealing outperforms all of them over the whole range of aperiodic load, with the largest gain at high load [2, Sec. 5.9, Fig. 5.25].

Table II summarises this qualitatively. The Polling Server is the easiest of all of them to implement and to analyse, but it is also the only one that never preserves unused capacity, which is why it gives the worst response time of the group [2, Sec. 5.10].

VI. CRITICAL EVALUATION

The main strength of the Polling Server is exactly its simplicity: it asks nothing of the scheduler beyond what periodic tasks already get, and its effect on the rest of the task set can be checked with the same RM-based tools [1], [2, Sec. 4.3]. The parameters C_{ps} and T_{ps} give a designer two numbers that directly say how much processor time is reserved for aperiodic work and how often that reservation is renewed [2, Sec. 5.3.2].

The most important limitation is the capacity-loss rule itself: an aperiodic request that arrives just after a poll has found an empty queue gets none of the benefit of the reservation until the next poll, so the average response time is worse than for any of the servers in Section V that preserve capacity in some form [2, Sec. 5.9]. Timing inaccuracy is a second, more practical risk – on a real platform, timer resolution, interrupt latency and context-switch cost all affect when a poll actually happens, and a poll that fires a little late or early changes exactly which requests are seen as “pending” [2, Sec. 10.7].

Like every server discussed here, the Polling Server does not solve overload: it bounds how much service aperiodic work receives, but if aperiodic demand keeps exceeding C_{ps} per period, the queue grows and response time degrades regardless [2, Ch. 9]. If periodic and aperiodic tasks share resources such as locks, buffers or peripherals, blocking and priority inversion

become relevant too, and protocols such as priority inheritance are needed to keep the timing analysis valid [2, Sec. 7.6] [7].

Finally, the UPPAAL model in Section IV is a single, illustrative scenario with poll instants at $g=2, 5, 10$, which are not evenly spaced; it is not a parametric, looping server template, and only two aperiodic requests are modelled, each occurring once. The four properties checked are existential ($E\Diamond$), so they show that a behaviour can occur but say nothing about how often, which would need statistical model checking.

VII. EMBEDDED-SYSTEM APPLICATION DISCUSSION

A typical fit for the Polling Server is a small embedded controller that already runs a simple, cyclic-executive style scheduler, where adding a Sporadic-Server-style replenishment timer would be disproportionate. Consider a sensor node that periodically samples and transmits data, but occasionally also has to answer a configuration or status request from a gateway: serving that request within one extra period is usually acceptable, and the Polling Server needs nothing beyond the periodic task’s own activation logic.

A second example is a low-end motor controller where the periodic control loop must never be disturbed, and the only aperiodic activity is an occasional operator command (start, stop, mode change) entered through a button or a slow serial link. Here the Polling Server’s worst case, waiting up to $T_{ps} + C_{ps}$, is small in absolute terms and well within what a human operator would notice.

The approach is less suitable when aperiodic events are safety-relevant or frequent – for example alarm signals in a fire- and gas-detection system – or when the period T_{ps} that would be needed for good aperiodic response is so short that it adds noticeable overhead to the periodic schedule. In those cases the Sporadic Server, Priority Exchange or Slack Stealing from Section V are a better match.

VIII. OPEN ISSUES AND POSSIBLE EXTENSIONS

One open issue is the gap between the model in Section IV and a real RTOS implementation: a practical follow-up could measure how timer granularity and interrupt latency shift the actual poll instants away from $g=2, 5, 10$, and how much that changes the observed worst case. A second direction is to make the model parametric in T_{ps} and C_{ps} , with a loop back from Idle to WaitFirstPoll, so that the same template could be reused to check other budget/period combinations rather than only the one scenario shown here.

A natural extension is a side-by-side model: add a second server template implementing the Sporadic Server’s replenishment rule, and run both on the same Job1/Job2 scenario to see directly how much earlier Job2 would be served. On the dynamic-priority side, the Total Bandwidth Server [2, Sec. 6.4] [6] plays a role similar to the one the Sporadic Server plays for fixed priorities, and could be modelled the same way as a point of comparison. Finally, as in Section VI, any of these extensions should eventually be considered together with resource-access protocols, since blocking on a shared resource can invalidate the timing assumptions used here.

IX. AI USAGE AND PROTOCOL

AI assistance was used for planning the structure of this paper, for language review, and for formatting the LaTeX source, the figures and the tables. It was not used as a technical source: the definitions, the schedulability argument, and the UPPAAL model and its verification results are the author's own work, based on the cited references – in particular Buttazzo's textbook [2], the original Sporadic Server paper [4] used for comparison, and the UPPAAL literature [8]–[10]. The AI interactions are documented separately in the AI usage protocol submitted with this paper.

X. CONCLUSION

The Polling Server is the simplest of the fixed-priority aperiodic servers, and the price for that simplicity is the capacity-loss rule: unused budget is gone as soon as a poll finds the queue empty, which gives it the worst guaranteed and average response time among the mechanisms in Section V. Its strength is that it needs nothing beyond what a periodic task already has, and can be analysed with the same RM-based tools. The UPPAAL model in Section IV, built around two aperiodic requests at $t=1$ and $t=6$ against polls at $g=2, 5, 10$, reproduces this trade-off directly: Job1 is served immediately, while Job2 has to wait from $t=6$ to $t=10$ because the budget was already lost at the second poll, and four reachability properties confirm that this is an execution the model actually reaches.

The Polling Server is well suited to small embedded systems with a predictable periodic core and infrequent, non-critical aperiodic events, where waiting an extra period is acceptable and where the implementation cost of a Sporadic-Server-style replenishment timer would not be justified. It is a poor fit for safety-relevant or frequent aperiodic work, where the Sporadic Server, Priority Exchange, or Slack Stealing offer a better balance between responsiveness and implementation effort.

REFERENCES

- [1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.
- [3] J. P. Lehoczky, L. Sha, and J. K. Strosnider, "Enhanced aperiodic responsiveness in hard real-time environments," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, San Jose, CA, USA, 1987, pp. 261–270.
- [4] B. Sprunt, L. Sha, and J. P. Lehoczky, "Aperiodic task scheduling for hard-real-time systems," *Real-Time Systems*, vol. 1, no. 1, pp. 27–60, 1989.
- [5] J. P. Lehoczky and S. Ramos-Thuel, "An optimal algorithm for scheduling soft-aperiodic tasks in fixed-priority preemptive systems," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, Phoenix, AZ, USA, 1992, pp. 110–123.
- [6] M. Spuri and G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179–210, 1996.
- [7] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990.
- [8] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

- [9] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.
- [10] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Berlin, Germany: Springer, 2004, pp. 200–236.